

Data Structure Level 1

Singly Linked List Data Structures

The background of the slide features a complex, abstract network of thin, dark lines crisscrossing a light gray background. In the lower portion of the image, there are dark silhouettes of several people, some standing and some leaning over, suggesting a collaborative work environment or a meeting. A large, dark gray rectangular area is overlaid on the right side of the image, serving as a backdrop for the text.

Data Structure

Level 1

Singly Linked List Data Structure



Definition

A Singly Linked List consists of nodes where:

- Each node contains:
 - A value (data)
 - A pointer to the next node (next)
- The head points to the first node
- The last node points to NULL, indicating the end of the list



Explanation

- ◆ Unlike arrays, linked lists do not require contiguous memory.
- ◆ Insertion and deletion are more flexible because only pointers need to be updated.
- ◆ Traversal is done by following the next pointers from the head node.

Procedures (Creating a Singly Linked List)

- 1 Define a Node class with value and next
- 2 Create node pointers → Node* Node1, Node2, Node3
- 3 Allocate memory for each node using new
- 4 Assign values to each node → Node1->value = 1
- 5 Connect nodes → Node1->next = Node2
- 6 Set the last node's next to NULL
- 7 Traverse from head and print values.

Example

Nodes: 1 → 2 → 3 → NULL

➡ Traversing the list prints:

1
2
3



Note

The head pointer is essential to access the list. Losing the head pointer means losing the entire list.

Warning

Accessing a node without proper allocation (e.g., uninitialized pointer) leads to runtime errors.



Comparison

■ Linked List

- ✓ Dynamic size
- ✓ Efficient insertions/deletions anywhere
- ✓ Requires extra memory for pointers

■ Array

- ✓ Fixed size (unless using vector)
- ✓ Fast random access
- ✓ Insertions/deletions require shifting elements

Summary

- ✓ Singly linked list stores nodes with value + next pointer
- ✓ Head points to the first node
- ✓ Traversal is done sequentially
- ✓ Supports dynamic memory allocation and flexible insertion/deletion

Thank you 😊